

What can Authorization Handler Return

Success - context.Succeed()

Failure - context.Fail()

Nothing - Task.CompletedTask

When you have multiple handlers

- **Failure takes precedence over success**
- **When one of the handlers return failure, the policy fails even if the other handlers return success**

Към момента имаме

Handler-1: [CanEditOnlyOtherAdminRolesAndClaimsHandler](#)

Handler-2: [SuperAdminHandler](#)

И двата унаследяват изискване за ауторизация от един `ManageAdminRolesAndClaimsRequirement`

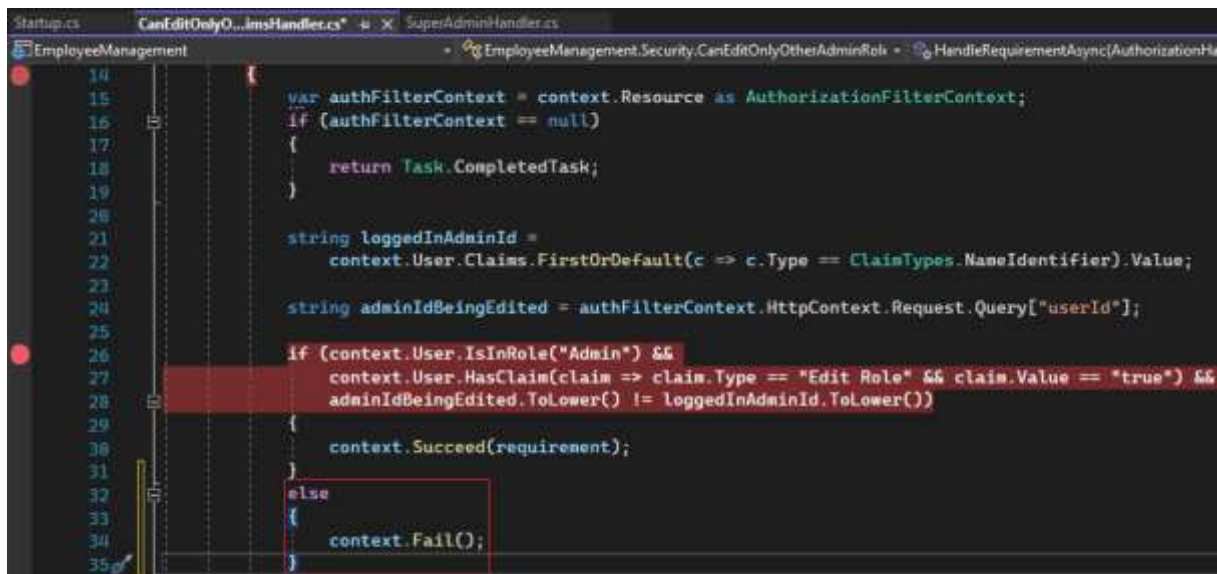
В кодовете им при успех се извиква

```
context.Succeed(requirement);
```

Никъде нямаме код как да се процедира при конкретен failure.

В Handler-1 в if-блока поставяме нов BreakPoint + else, който да прихване failure:

```
else
{
    context.Fail();
}
```



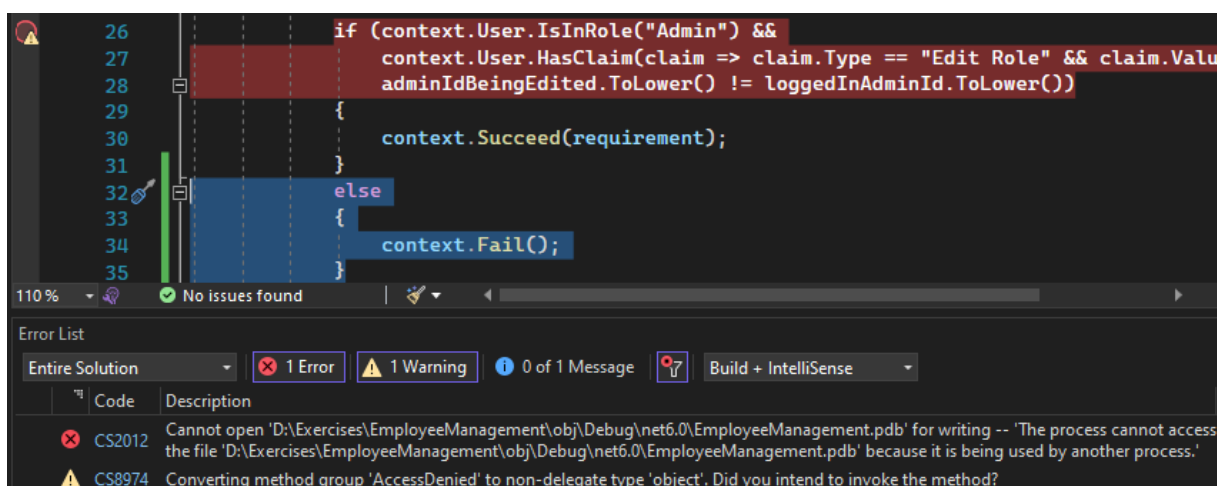
```
14
15
16 var authFilterContext = context.Resource as AuthorizationFilterContext;
17 if (authFilterContext == null)
18 {
19     return Task.CompletedTask;
20 }
21
22 string loggedInAdminId =
23     context.User.Claims.FirstOrDefault(c => c.Type == ClaimTypes.NameIdentifier).Value;
24
25 string adminIdBeingEdited = authFilterContext.HttpContext.Request.Query["userId"];
26
27 if (context.User.IsInRole("Admin") &&
28     context.User.HasClaim(claim => claim.Type == "Edit Role" && claim.Value == "true") &&
29     adminIdBeingEdited.ToLower() != loggedInAdminId.ToLower())
30 {
31     context.Succeed(requirement);
32 }
33 else
34 {
35     context.Fail();
36 }
```

При Debug като SuperAdmin -> ListUsers ->Edit->Manage Roles ->

Handler-1 BreakPoint-1, BreakPoint-2, но понеже потребителят не отговаря на нито едно от условията, отива в else -> .Fail();

После прескача на Handler-2, BreakPoint3, където удовлетворява условието за Super Admin, но при Continue -> Access Denied. Това е, защото вече е извикан един Fail. Когато това се случи, независимо дали по-нататък ще се удовлетвори някакво условие, Fail е с предимство и ще се изпълни той.

В моята версия прескача направо на BreakPoint-3, дава ми достъп, но и интересен вид грешка:



```
26
27
28 if (context.User.IsInRole("Admin") &&
29     context.User.HasClaim(claim => claim.Type == "Edit Role" && claim.Value == "true") &&
30     adminIdBeingEdited.ToLower() != loggedInAdminId.ToLower())
31 {
32     context.Succeed(requirement);
33 }
34 else
35 {
36     context.Fail();
37 }
```

Error List

Code	Description
CS2012	Cannot open 'D:\Exercises\EmployeeManagement\obj\Debug\net6.0\EmployeeManagement.pdb' for writing -- 'The process cannot access the file 'D:\Exercises\EmployeeManagement\obj\Debug\net6.0\EmployeeManagement.pdb' because it is being used by another process.'
CS8974	Converting method group 'AccessDenied' to non-delegate type 'object'. Did you intend to invoke the method?

Като правило .Fail не се използва в Handler, за да не се провали алгоритъма, ако друг Handler е успешен. Използваме Fail за Handler, само ако искаме да гарантираме отказване на достъпа, дори някой от другите Handler-и е успешен.

Трябва да извикаме .Succeed и да не използваме .Fail

When you have multiple handlers

- **Failure takes precedence over success**
- When one of the handlers return failure, the policy fails even if the other handlers return success
- **In general, do not return failure from a handler, as other handlers for the same requirement may succeed**
- Only return an explicit failure, to guarantee the failure of the policy even when the other handlers succeed
- **If none of the handlers return an explicit success the policy will not succeed**
- For a policy to succeed
 - An explicit **success** must be returned from one of the handlers, AND
 - No other handler must return an explicit **failure**

Ако искаме подредбата по подразбиране да се промени, задаваме

```
services.AddAuthorization(options =>
{
    options.AddPolicy("EditRolePolicy", policy =>
        policy.AddRequirements(new ManageAdminRolesAndClaimsRequirement()));

    options.InvokeHandlersAfterFailure = false;

});
```

Set **InvokeHandlersAfterFailure** to false

- By default, all the handlers are called, irrespective of what a handler returns
 - Success
 - Failure
 - Nothing
- If you do not want the rest of the handlers to be called, when a failure is returned, set **InvokeHandlersAfterFailure** property to **false**. The default is true

```
services.AddAuthorization(options =>
{
    options.AddPolicy("EditRolePolicy", policy =>
        policy.AddRequirements(new ManageAdminRolesAndClaimsRequirement()));

    options.InvokeHandlersAfterFailure = false; ←
});
```

В този случай (който НЕ е по подразбиране в моята версия), ако даден Handler се провали и се превключи на следващ, който изпълни изискванията, операцията се изпълнява.

```
services.AddAuthorization(options =>
{
    options.AddPolicy("DeleteRolePolicy",
        policy => policy.RequireClaim("Delete Role"));

    options.AddPolicy("EditRolePolicy",
        policy => policy.AddRequirements(new ManageAdminRolesAndClaimsRequirement()));

    options.InvokeHandlersAfterFailure = false;

    options.AddPolicy("AdminRolePolicy",
        policy => policy.RequireRole("Admin"));
});
```

Сега в Handler-1 оставяме само 1-я генерален BreakPoint, а втория пред if-блока го отстраняваме. В Handler-2 имаме един BreakPoint-2. Даваме Debut и BP1 не е удовлетворен, защото сме Super Admin -> Continue не стига дори до BP2, който отговаря на условията, а прехвърля директно на Access Denied. Това е заради реда код, който току-що поставихме options.InvokeHandlersAfterFailure = false;

затова го премахваме, както

```
else
{
    context.Fail();
}
```